

Tanaka Certificates: Weekly Research Report

Filip Morawiec

6 July 2026

Abstract

This week I contributed by implementing a multidimensional verifier for ReLU neural networks (with last layer being piecewise quadratic to ensure completeness) and proving it's correctness. After meeting with Greg, he recommended that I should focus on proving the correctness of the ReLU NN region discovery algorithm and algebraically working through an example 2D SDE verification, which I did and present here. I also worked on training a certificate for this problem - following the techniques used by Greg in his previous paper - but I failed to satisfy all the constraints a certificate should have.

1 Multidimensional verifier

1.1 Problem setup

Consider a multi-dimensional stochastic differential equation

$$dX_t = f(X_t) dt + g(X_t) dW_t \quad (1)$$

on a domain $D \subset \mathbb{R}^d$. For a reach-avoid problem, let $X_0 \subseteq D$ be the initial set, $X_u \subseteq D$ the unsafe set, and $X_\tau \subseteq D$ the target. We seek a certificate V and constants $\alpha < \beta$ and $\varepsilon > 0$ such that

1. boundary condition holds:

$$\sup_{x \in X_0} V(x) \leq \alpha \quad \text{and} \quad \inf_{x \in X_u} V(x) \geq \beta \quad (2)$$

2. generator condition holds:

$$\mathcal{L}V(x) \leq -\varepsilon \quad \text{for } x \in \{V \leq \beta\} \setminus X_\tau, \quad (3)$$

where, at smooth points,

$$\mathcal{L}V(x) = V'(x)f(x) + \frac{1}{2}V''(x)g(x)^2. \quad (4)$$

The process is stopped after reaching the target or unsafe set: once the reach-avoid property has been achieved or violated, its later trajectory is irrelevant.

We model a certificate V as:

$$V(x) = \sum_{k=1}^m \lambda_k \phi(z_k(x)) + c, \quad (5)$$

$$z(x) = \text{NN}(x) = W_L \text{ReLU}(\dots \text{ReLU}(W_1 x + b_1) \dots) + b_L, \quad (6)$$

$$\phi = \text{PiecewiseQuadratic1D}. \quad (7)$$

This way, because we include a piecewise quadratic function ϕ in the last layer, we cover a certain failure case related to the curvature term $V''(x)$ in the generator condition (3).

2 Correctness proof of 1D ReLU NN region discovery

Consider the following Algorithm 1 which discovers the linear cells of a one-dimensional ReLU network.

Listing 1: Discover one-dimensional linear cells of a ReLU network

```

1 def discover_cells(layers):
2     """
3     The network has a one-dimensional input and a one-dimensional output, but hidden
4     layers can be multi-dimensional:
5     >>> layers[i] = (W_i, b_i)
6     >>> W_i.shape = (n_{i+1}, n_i)
7     >>> b_i.shape = (n_{i+1},)
8     """
9
10    # A cell is (lower, upper, slope, intercept).
11    cells = [(-inf, inf, 1, 0)]
12
13    for W, b in layers:
14        new_cells = []
15        for lower, upper, slope, intercept in cells:
16            # key observation: in a cell C_prev(l, u, slope, intercept), we have
17            #   z_prev = slope * x + intercept, so the linear function
18            # in the current cell C(?, ?, alpha, beta) will be
19            #   z = W * z_prev + b = (W * slope) * x + (W * intercept + b)
20            alpha = W * slope
21            beta = W * intercept + b
22
23            roots = {
24                -beta[j] / alpha[j]
25                for j in range(len(beta))
26                if alpha[j] != 0
27                and lower < -beta[j] / alpha[j] < upper
28            }
29            boundaries = [lower, *sorted(roots), upper]
30
31            for left, right in consecutive_pairs(boundaries):
32                x = choose_point(left, right)
33                active = alpha * x + beta > 0
34                new_slope = where(active, alpha, 0)
35                new_intercept = where(active, beta, 0)
36                new_cells.append(
37                    (left, right, new_slope, new_intercept)
38                )
39
40        cells = new_cells
41
42    return cells

```

It discovers the linear cells of a one-dimensional ReLU network, with:

$$\text{NN}(x) = W_L \text{ReLU}(\dots \text{ReLU}(W_1 x + b_1) \dots) + b_L. \quad (8)$$

To prove the correctness algorithms, the key step is to show, given a linear cell of the network up to layer $m - 1$, how to discover the linear cells of the network up to layer m . As a precondition, we already have a linear cell of the network when we begin: $\mathcal{C}(-\infty, \infty, 1, 0)$ represents the entire real line, with slope 1 and intercept 0.

3 2D SDE verification example

4 Certificate training